

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Patiala, Punjab

RESQAI

AI-Assisted Disaster Response System

Humanitarian Message Classification and Crisis Information Analysis

PROJECT REPORT

Submitted in partial fulfillment of the requirements
for the academic project

Team APEX

Dhwani Agarwal 1024031024

Sanchita Jain 1024031020

Yuvakshi Sood 1024031025

Department of Computer Science
Thapar Institute of Engineering and Technology
Patiala

2026

Abstract

Natural disasters and humanitarian emergencies generate large amounts of information through social media, emergency messages, and other communication channels. During such situations, manually examining every incoming message is difficult because the information is high-volume, unstructured, and heterogeneous.

RESQAI is an AI-assisted disaster-response system designed to automatically classify disaster-related humanitarian messages and extract useful information for subsequent response activities. The current machine learning prototype uses the HumAID dataset and a fine-tuned BERTweet model for ten-class humanitarian message classification.

The final BERTweet classifier achieved 78.67% test accuracy and 78.27% weighted F1-score. To address ambiguous cases, a hybrid architecture incorporates Gemini as a fallback analyzer. Messages with low BERTweet confidence or messages classified as `not_humanitarian` are forwarded to Gemini for contextual analysis and structured extraction of category, humanitarian relevance, urgency, request for help, location, and explanation.

This report additionally presents the software engineering design of RESQAI through functional and non-functional requirements, a Use Case Diagram, UML system architecture, Activity Diagram, Class Diagram, and Software Development Life Cycle (SDLC) model. The complete application involving user interfaces, incident management, rescue coordination, and map integration is described as the proposed system architecture, while the BERTweet and Gemini classification pipeline represents the current implemented prototype.

Keywords: Disaster Response, Humanitarian Classification, BERTweet, NLP, Gemini, Artificial Intelligence, Social Media, Machine Learning, RESQAI.

Contents

Abstract	1
1 Introduction	6
1.1 Background and Motivation	6
1.2 Problem Statement	6
1.3 Objectives	6
1.4 Scope	7
2 Requirements Analysis	8
2.1 Stakeholders	8
2.2 Functional Requirements	8
2.3 Non-Functional Requirements	9
3 System Design	10
3.1 Overview	10
3.2 Use Case Diagram	10
3.3 UML System Architecture	11
3.4 Activity Diagram	13
3.5 Class Diagram	14
3.6 Software Development Life Cycle	15
3.6.1 Requirement Analysis	16
3.6.2 System Design	16
3.6.3 Implementation	16
3.6.4 Testing	17
3.6.5 Deployment	17
3.6.6 Maintenance and Improvement	17
4 Dataset and Methodology	18
4.1 HumAID Dataset	18
4.2 Humanitarian Categories	18
4.3 Data Preprocessing	19

4.4	BERTweet Model	19
4.5	Fine-Tuning	20
4.6	Hybrid BERTweet–Gemini Architecture	20
4.7	Gemini Structured Analysis	20
5	Implementation	22
5.1	Technology Stack	22
5.2	Model Inference Pipeline	22
5.3	Example Inference	23
6	Results and Evaluation	24
6.1	Model Performance	24
6.2	Accuracy	24
6.3	Weighted F1-Score	24
6.4	Hybrid Model Evaluation Status	25
7	Discussion	26
7.1	Why BERTweet?	26
7.2	Why Not Use Gemini Alone?	26
7.3	Why Use a Hybrid Architecture?	26
8	Error Analysis and Improvement	28
8.1	Observed Classification Challenges	28
8.2	Potential Accuracy Improvements	28
9	Advantages, Limitations and Safety	30
9.1	Advantages	30
9.2	Limitations	30
9.3	Safety Considerations	31
10	Future Scope	32
10.1	User-Facing Interface	32
10.2	Administrator Dashboard	32
10.3	Rescue-Team Dashboard	33
10.4	Map Integration	33
10.5	Multilingual Disaster Analysis	33
10.6	Improved Urgency Detection	33
10.7	Real-Time Data Streams	33
10.8	Continuous Model Improvement	34
11	Conclusion	35

List of Figures

3.1	Use Case Diagram for RESQAI	11
3.2	UML System Architecture of RESQAI	12
3.3	Activity Diagram for RESQAI	13
3.4	Class Diagram for RESQAI	14
3.5	Software Development Life Cycle for RESQAI	16

List of Tables

4.1	HumAID Dataset Split	18
5.1	Technology Stack	22
6.1	BERTweet Performance	24
7.1	Roles of BERTweet and Gemini	27

1. Introduction

1.1 Background and Motivation

During natural disasters such as earthquakes, floods, cyclones, wildfires, and other emergencies, people frequently communicate through social media and digital platforms. These messages can contain information about damaged infrastructure, missing individuals, injuries, evacuation requirements, urgent resource requirements, and volunteer or donation activities.

The large volume of messages creates a challenge for disaster management authorities. Important information can be mixed with irrelevant conversations, general observations, or duplicate information. An automated system capable of identifying and categorizing humanitarian information can therefore act as an information-filtering layer for emergency response.

RESQAI focuses on this problem by applying Natural Language Processing (NLP) and Transformer-based language models to disaster-related messages. The system classifies messages into predefined humanitarian categories and uses a secondary reasoning model for uncertain cases.

1.2 Problem Statement

The central problem addressed by RESQAI is:

How can disaster-related messages be automatically identified, classified, and converted into structured humanitarian information to support effective disaster-response operations?

The system aims to reduce the amount of manual filtering required by providing an automated classification layer that can identify the nature of incoming disaster information.

1.3 Objectives

The main objectives of RESQAI are:

1. To develop an automated classification system for disaster-related messages.

2. To fine-tune BERTweet on the HumAID humanitarian message dataset.
3. To classify messages into ten predefined humanitarian categories.
4. To generate a confidence score for each BERTweet prediction.
5. To introduce Gemini as a fallback mechanism for low-confidence or potentially misclassified messages.
6. To generate structured humanitarian information including urgency, location, help requirements, and explanation.
7. To design a complete disaster-response software architecture containing reporter, administrator, and rescue-team components.
8. To provide a foundation for future integration with maps, dashboards, notifications, and incident management.

1.4 Scope

The current implemented scope primarily covers the machine learning and NLP pipeline:

- HumAID dataset preparation.
- Text preprocessing and tokenization.
- BERTweet fine-tuning.
- Ten-class humanitarian classification.
- Confidence score generation.
- Gemini fallback analysis.
- Structured output generation.
- Validation and test evaluation.

The complete user-facing application, database-backed incident management, rescue-team coordination, map integration, and notifications form the proposed system scope and future integration work.

2. Requirements Analysis

2.1 Stakeholders

The main stakeholders of RESQAI are:

1. **Reporter/Citizen:** Submits a disaster message or SOS request and may provide location information.
2. **Administrator/Disaster Authority:** Reviews incoming incidents, classifications, urgency information, and assigns rescue teams.
3. **Rescue Team:** Receives assigned incidents, views locations, updates task status, and marks incidents as resolved.
4. **AI Classification Module:** Processes messages, performs classification, generates confidence scores, and invokes Gemini when necessary.
5. **External Services:** APIs such as Gemini and map services used by the proposed application.

2.2 Functional Requirements

The proposed RESQAI system should provide the following functional requirements:

1. The system shall allow a reporter to submit a disaster-related message.
2. The system shall optionally accept the reporter's location.
3. The system shall preprocess and tokenize the message.
4. The system shall classify the message using the BERTweet model.
5. The system shall generate a confidence score for the prediction.
6. The system shall invoke Gemini when BERTweet confidence is below the defined threshold or when the predicted class is `not_humanitarian`.

7. The system shall generate structured information from the classification pipeline.
8. The administrator shall be able to view and filter incidents.
9. The administrator shall be able to assign incidents to rescue teams.
10. Rescue teams shall be able to view assigned incidents and update their status.
11. The system shall support location visualization in the proposed map-integrated application.
12. The system should provide incident status information to relevant users.

2.3 Non-Functional Requirements

- **Accuracy:** The classification system should provide reliable humanitarian classification.
- **Reliability:** The system should continue to provide classification results when the primary model is available.
- **Scalability:** The proposed application should be capable of handling increasing numbers of incoming messages.
- **Security:** User information, API credentials, and incident data should be protected.
- **Usability:** The reporter interface should be simple enough to use during an emergency.
- **Maintainability:** The ML model and application components should be independently maintainable.
- **Availability:** The complete system should be designed for continuous operation during emergencies.

3. System Design

3.1 Overview

The RESQAI system consists of a user interaction layer, application layer, AI classification layer, data layer, and external service integration.

The current prototype implements the AI classification layer. The complete architecture additionally defines the interaction between reporters, administrators, rescue teams, application services, database services, and external APIs.

3.2 Use Case Diagram

The Use Case Diagram represents the interaction between the major actors and the RESQAI system.

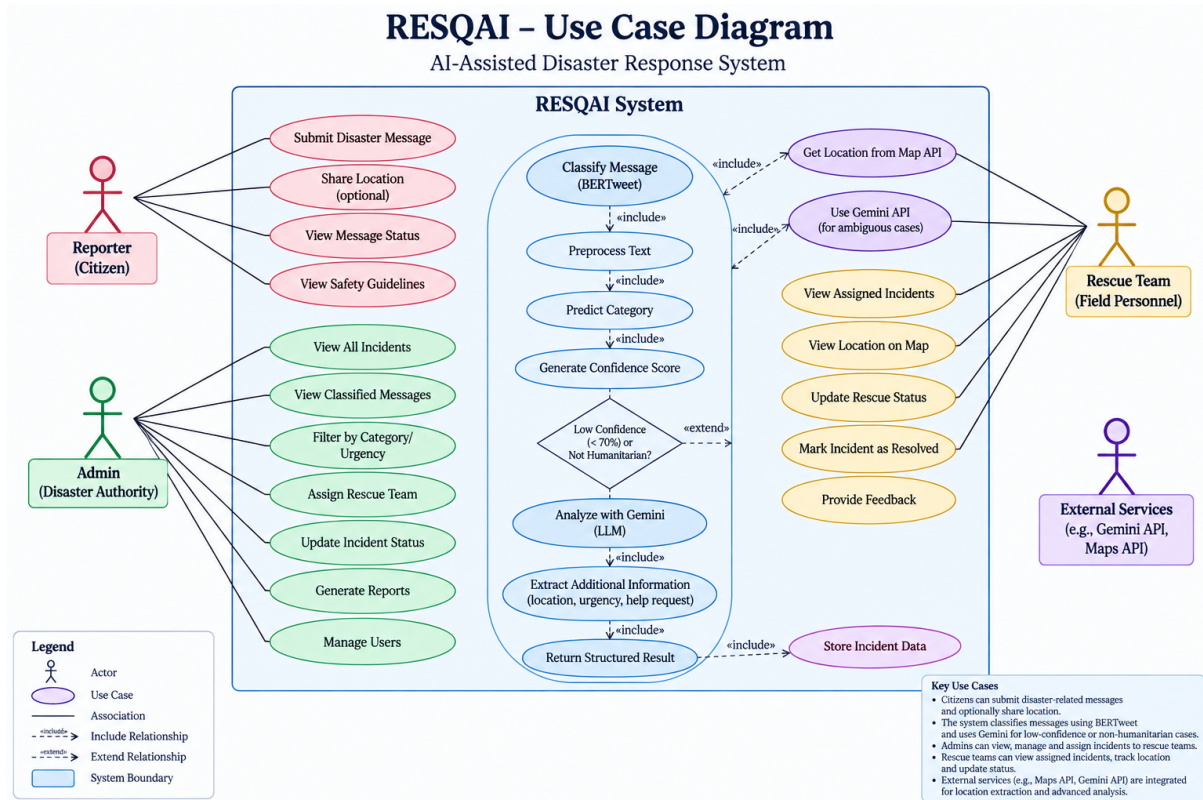


Figure 3.1: Use Case Diagram for RESQAI

The primary actor interactions are:

- The **Reporter** submits a disaster message, optionally shares location, and views the status of submitted information.
- The **Administrator** views incidents, examines classifications, filters information, assigns rescue teams, and updates incident information.
- The **Rescue Team** views assigned incidents, checks locations, updates rescue status, and marks incidents as resolved.
- The **AI Classification Pipeline** performs preprocessing, BERTweet classification, confidence evaluation, and Gemini analysis when required.
- **External APIs** support functionality such as Gemini analysis and proposed map services.

3.3 UML System Architecture

The high-level UML/component-oriented architecture represents the major modules and their interactions.

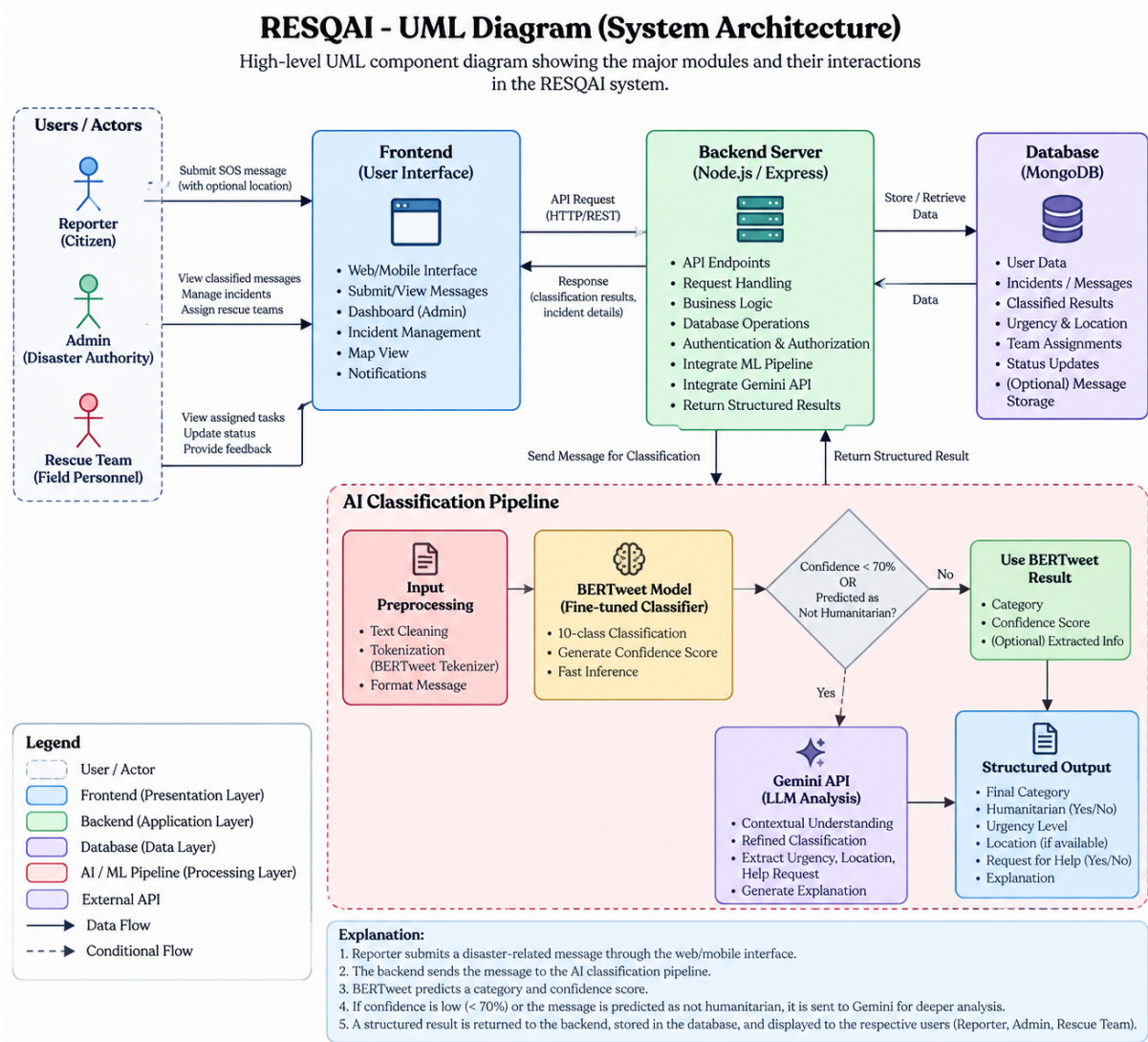


Figure 3.2: UML System Architecture of RESQAI

The architecture contains the following major components:

1. **Frontend:** Provides the user-facing interface for reporters, administrators, and rescue teams.
2. **Backend Server:** Handles application requests, business logic, database operations, and integration with the AI pipeline.
3. **Database:** Stores user information, incidents, classification results, assignments, and status information in the proposed complete system.
4. **AI Classification Pipeline:** Contains the BERTweet classifier, confidence layer, and Gemini fallback.
5. **External Services:** Includes the Gemini API and proposed map APIs.

The AI pipeline follows:

$$\text{Message} \rightarrow \text{BERTweet} \rightarrow \text{Confidence Check} \rightarrow \begin{cases} \text{BERTweet Result,} & \text{high confidence} \\ \text{Gemini Analysis,} & \text{low confidence / not humanitarian} \end{cases} \quad (3.1)$$

3.4 Activity Diagram

The Activity Diagram represents the flow of a message through the RESQAI system.

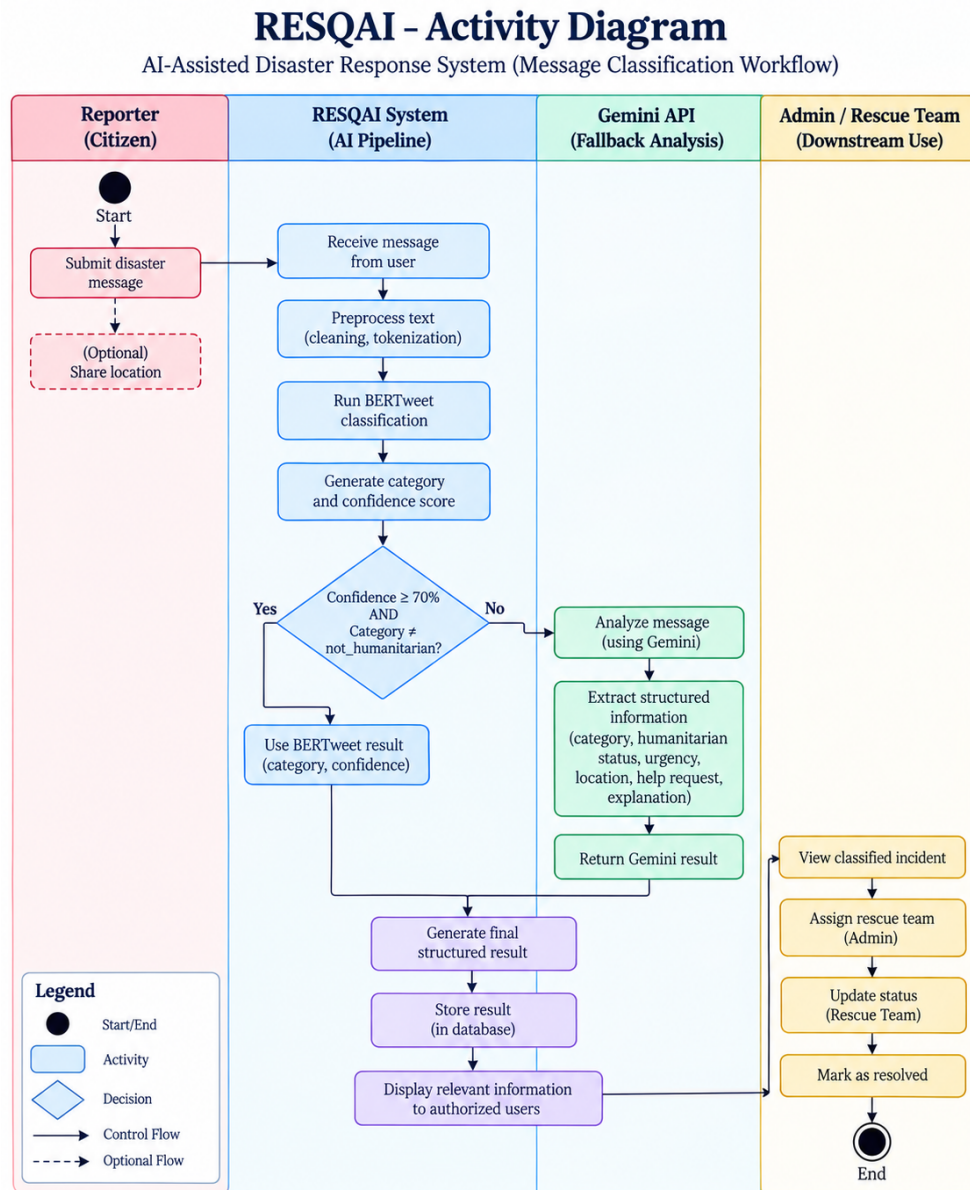


Figure 3.3: Activity Diagram for RESQAI

The activity begins when a reporter submits a message. The message is preprocessed and

passed to BERTweet. BERTweet generates a category and confidence score.

If the confidence is at least 70% and the prediction is not `not_humanitarian`, the BERTweet prediction is used directly. Otherwise, the message is forwarded to Gemini for contextual analysis. The resulting structured information can then be stored and displayed to the relevant users in the complete application.

3.5 Class Diagram

The Class Diagram describes the main entities and software components required by the proposed RESQAI application.

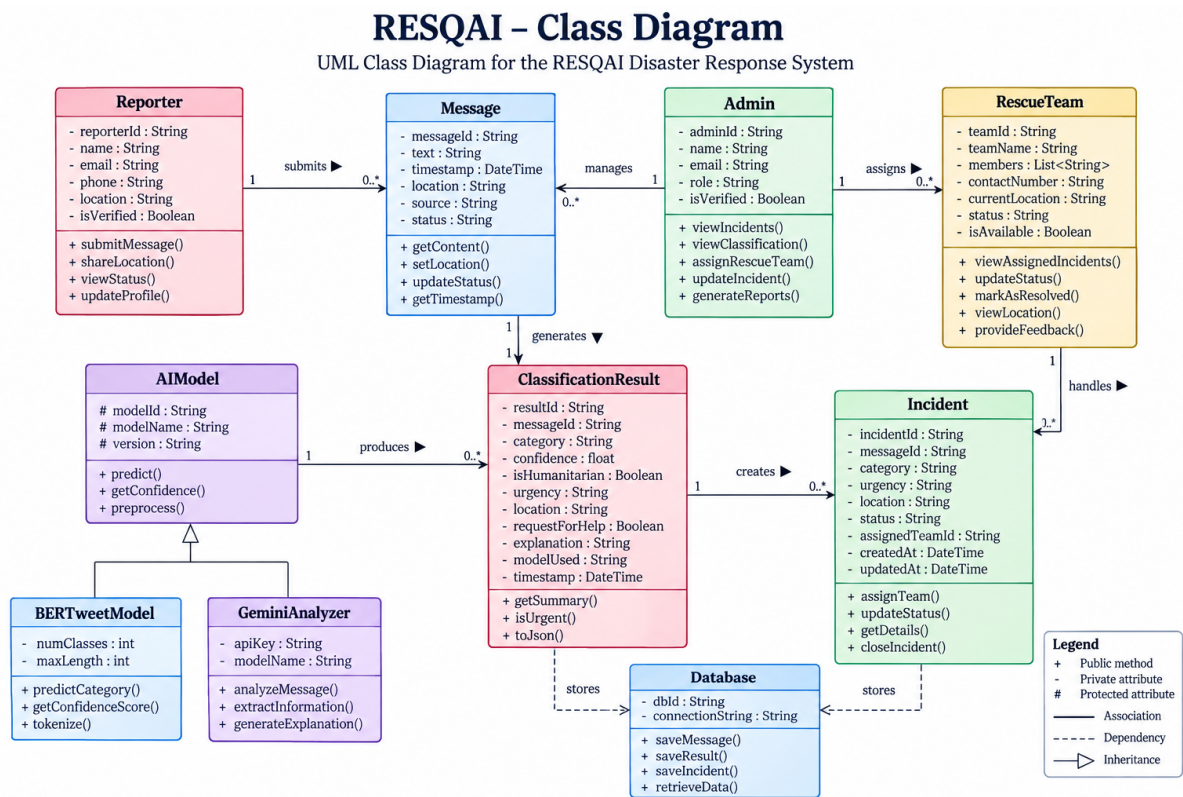


Figure 3.4: Class Diagram for RESQAI

The major classes include:

- **Reporter:** Represents a citizen interacting with the system.
- **Message:** Represents the incoming disaster-related message.
- **ClassificationResult:** Stores the AI-generated category, confidence, urgency, location, and other information.
- **AIModel:** Represents the generic AI classification component.

- **BERTweetModel:** Represents the fine-tuned classification model.
- **GeminiAnalyzer:** Represents the fallback LLM analysis component.
- **Incident:** Represents a disaster-response case generated from relevant information.
- **Admin:** Represents the disaster authority managing incidents.
- **RescueTeam:** Represents field personnel assigned to incidents.
- **Database:** Represents persistent storage in the complete application.

3.6 Software Development Life Cycle

RESQAI follows a structured SDLC approach in which the system moves from requirement identification through design, implementation, testing, deployment, and continuous improvement.

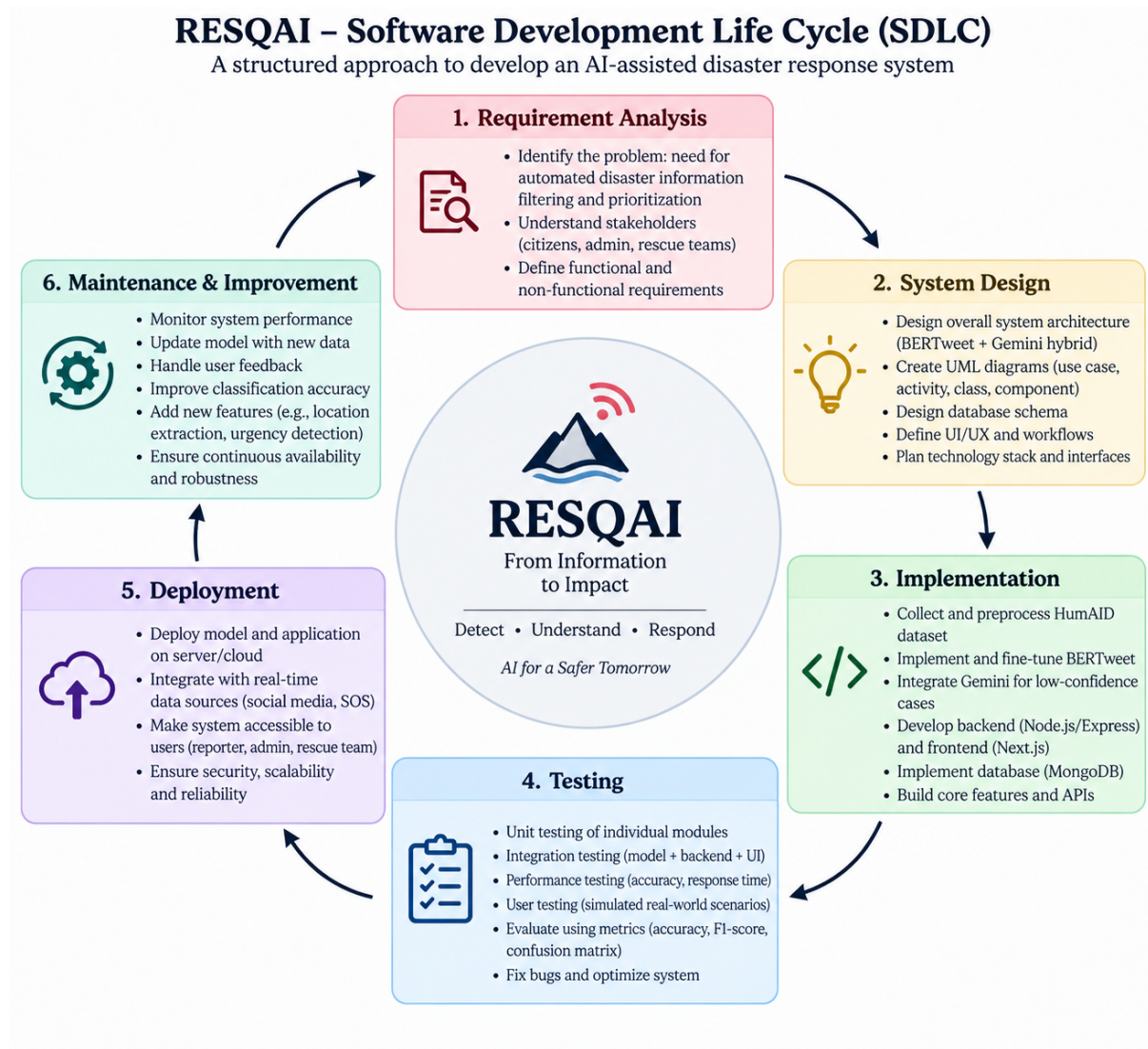


Figure 3.5: Software Development Life Cycle for RESQAI

3.6.1 Requirement Analysis

The problem of filtering and prioritizing disaster-related information is identified. Stakeholders, system requirements, and constraints are analyzed.

3.6.2 System Design

The architecture of the AI pipeline and proposed application is designed. UML diagrams, database structure, user workflows, and technology components are defined.

3.6.3 Implementation

The HumAID dataset is prepared and the BERTweet model is fine-tuned. The Gemini fallback is integrated into the classification prototype. The proposed full application includes frontend,

backend, database, and external API components.

3.6.4 Testing

The model is evaluated using validation and test data. Accuracy, weighted F1-score, and confusion patterns are analyzed. The application components can subsequently be tested using unit, integration, and scenario-based testing.

3.6.5 Deployment

The complete system can be deployed on a server or cloud platform and connected to real-time data sources and user interfaces.

3.6.6 Maintenance and Improvement

Future maintenance includes retraining with new data, improving classification accuracy, tuning the confidence threshold, fixing software issues, and adding capabilities such as multilingual classification and improved location extraction.

4. Dataset and Methodology

4.1 HumAID Dataset

The HumAID dataset is a humanitarian information dataset containing disaster-related social-media messages. It provides manually annotated humanitarian categories and is suitable for training models for disaster-related information classification.

The dataset used in the project contains:

Table 4.1: HumAID Dataset Split

Split	Number of Messages
Training	53,531
Validation	7,793
Test	15,160
Total	76,484

4.2 Humanitarian Categories

The model performs ten-class classification:

1. caution_and_advice
2. displaced_people_and_evacuations
3. infrastructure_and_utility_damage
4. injured_or_dead_people
5. missing_or_found_people
6. not_humanitarian
7. other_relevant_information
8. requests_or_urgent_needs

9. rescue_volunteering_or_donation_effort
10. sympathy_and_support

4.3 Data Preprocessing

The textual input is converted into a format suitable for BERTweet. The BERTweet tokenizer is used to tokenize the messages.

A maximum sequence length of 128 tokens is used in the current implementation. Longer messages are truncated, while shorter messages are handled according to the tokenizer’s padding and attention-mask requirements.

Because BERTweet is pretrained on Twitter-style text, aggressive removal of social-media characteristics is avoided. Hashtags, informal expressions, and other contextual information may contain useful signals for disaster classification.

4.4 BERTweet Model

BERTweet is a Transformer-based language model pretrained on English Twitter data. The pretrained model is fine-tuned on the HumAID dataset for the specific ten-class classification task.

The processing pipeline is:

1. Input disaster message.
2. Tokenize the message using the BERTweet tokenizer.
3. Pass token representations through BERTweet.
4. Generate classification logits.
5. Apply Softmax to obtain class probabilities.
6. Select the class with the highest probability.
7. Record the corresponding confidence score.

The probability of class i is obtained using:

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}} \quad (4.1)$$

where z_i represents the logit for class i .

4.5 Fine-Tuning

Instead of training a language model from scratch, the pretrained BERTweet model is adapted to the humanitarian classification task.

During fine-tuning, the model receives a disaster-related message along with its ground-truth category. The model updates its parameters to reduce the classification loss.

This approach allows the project to benefit from the linguistic knowledge learned during BERTweet pretraining while adapting the model to disaster-response terminology.

4.6 Hybrid BERTweet–Gemini Architecture

BERTweet is used as the primary classifier. However, ambiguous messages can be difficult to classify using a fixed set of classes.

Therefore, a fallback mechanism was introduced.

The routing condition is:

$$\text{Gemini Used} = (\text{Confidence} < 0.70) \vee (\text{Prediction} = \text{not_humanitarian}) \quad (4.2)$$

Thus:

- High-confidence and acceptable BERTweet predictions are returned directly.
- Low-confidence predictions are sent to Gemini.
- Messages classified as `not_humanitarian` are also sent to Gemini because useful disaster information may be expressed indirectly.

4.7 Gemini Structured Analysis

Gemini is not trained as part of this project. Instead, its API is integrated as a secondary analysis component.

The output is constrained to structured fields:

- Category
- Humanitarian status
- Urgency
- Request for help

- Location
- Explanation

This makes the result easier for a future application to consume programmatically.

5. Implementation

5.1 Technology Stack

The current prototype uses:

Table 5.1: Technology Stack

Component	Technology
Programming Language	Python
ML Framework	PyTorch
Transformer Library	Hugging Face Transformers
Primary Model	BERTweet
Dataset	HumAID
Secondary Model	Gemini API
Structured Output	Pydantic schema
Development Environment	Jupyter / Google Colab
Version Control	Git and GitHub

The proposed complete application can additionally use a web frontend, backend server, MongoDB database, and map API.

5.2 Model Inference Pipeline

The inference process can be summarized as follows:

```
def predict_resqai(text, confidence_threshold=0.70):  
  
    bertweet_label, confidence = predict_bertweet(text)  
  
    send_to_gemini = (  
        confidence < confidence_threshold  
        or bertweet_label == "not_humanitarian"  
    )
```

```
if not send_to_gemini:
    return BERTweet_result

gemini_result = analyze_with_gemini(
    text=text,
    bertweet_prediction=bertweet_label,
    bertweet_confidence=confidence
)

return gemini_result
```

The implementation therefore uses BERTweet as the first-stage classifier and Gemini as a selective second-stage analyzer.

5.3 Example Inference

Consider the following message:

“We are trapped in the building and urgently need water and medical help.”

BERTweet can classify this as:

`requests_or_urgent_needs`

with a high confidence score. Since the confidence is above the routing threshold, Gemini does not need to be called.

For an ambiguous message, BERTweet may produce a lower confidence score. Such a message is passed to Gemini, which provides additional contextual information.

6. Results and Evaluation

6.1 Model Performance

The final fine-tuned BERTweet model produced the following results:

Table 6.1: BERTweet Performance

Metric	Value
Test Accuracy	78.67%
Test Weighted F1	78.27%
Best Validation Accuracy	78.40%
Best Validation Weighted F1	78.05%

6.2 Accuracy

Accuracy represents the proportion of test examples for which the predicted class matches the ground-truth class.

The obtained test accuracy is 78.67%, meaning that the final BERTweet classifier correctly classified approximately 79 out of every 100 unseen test messages on average.

6.3 Weighted F1-Score

F1-score combines precision and recall:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6.1)$$

Weighted F1 gives each class a contribution proportional to the number of samples belonging to that class.

The final weighted F1-score of the model is 78.27%.

6.4 Hybrid Model Evaluation Status

The BERTweet results above represent the formally evaluated machine learning classifier.

The Gemini component has been integrated and demonstrated as a fallback mechanism. However, an end-to-end hybrid accuracy over the entire test set has not been reported as a formal metric in the current implementation.

Therefore, the 78.67% accuracy should be described specifically as the **BERTweet test accuracy**, rather than as the accuracy of the complete hybrid system.

7. Discussion

7.1 Why BERTweet?

BERTweet was selected because the primary data consists of social-media-style English messages. The model was pretrained on Twitter data and is therefore well suited to the linguistic style of tweets, including informal expressions, hashtags, abbreviations, and short contextual messages.

More importantly, BERTweet can be fine-tuned specifically for the ten-class HumAID classification task. This gives the project a specialized classification model with a fixed and controlled output space.

7.2 Why Not Use Gemini Alone?

Gemini is a general-purpose large language model with strong contextual reasoning capabilities. However, the primary task of RESQAI is a fixed ten-class classification problem.

Using BERTweet as the primary classifier provides:

- A model specifically fine-tuned for the target task.
- A fixed ten-class output space.
- A direct probability distribution across the classes.
- A confidence score that can be used for routing.
- Reduced dependence on external API calls for straightforward cases.

Gemini is therefore used where additional contextual reasoning is useful rather than being called for every message.

7.3 Why Use a Hybrid Architecture?

The two models serve different roles:

Table 7.1: Roles of BERTweet and Gemini

	BERTweet	Gemini
Primary role	Classification	Fallback analysis
Training in project	Fine-tuned on HumAID	API integration
Output	Ten-class prediction	Structured contextual analysis
Confidence	Direct class probabilities	Not used as primary routing signal
Usage	Most messages	Low-confidence / not-humanitarian cases
Strength	Task specialization	Contextual reasoning

The hybrid design attempts to combine task-specific classification with contextual analysis.

8. Error Analysis and Improvement

8.1 Observed Classification Challenges

Some humanitarian messages do not explicitly request assistance. Instead, they provide situational information.

For example:

“Not sure if I slept thru aftershocks between 4-6am or if theyre just starting up again in Wellington?”

Such a message can be difficult to distinguish from `not_humanitarian` because it does not explicitly request help.

This illustrates the semantic overlap between `other_relevant_information` and `not_humanitarian`.

8.2 Potential Accuracy Improvements

Several approaches can be investigated to improve model performance:

1. **Class-weighted loss:** Assign greater importance to underrepresented classes during training.
2. **Learning-rate tuning:** Experiment with multiple learning rates and select the best configuration using the validation set.
3. **Epoch tuning:** Compare different training durations while monitoring validation performance.
4. **Weight-decay tuning:** Adjust regularization to reduce overfitting.
5. **Data quality analysis:** Identify duplicates, missing values, and possible data leakage.
6. **Targeted error analysis:** Analyze confusion between frequently confused classes.
7. **Input-length experiments:** Compare the current 128-token setting with longer sequence lengths where computational resources permit.

8. **Data augmentation:** Carefully generate additional examples for underrepresented classes and evaluate their impact.
9. **Confidence-threshold tuning:** Use validation data to determine an evidence-based routing threshold for Gemini.
10. **Hybrid evaluation:** Evaluate BERTweet alone against the complete BERTweet–Gemini pipeline using the same test set.

9. Advantages, Limitations and Safety

9.1 Advantages

- Automates classification of large numbers of disaster messages.
- Uses a model specifically adapted to social-media text.
- Provides a confidence score for classification.
- Uses Gemini selectively for ambiguous cases.
- Produces structured information suitable for application integration.
- Provides a foundation for incident management and rescue coordination.

9.2 Limitations

- The BERTweet classifier is not perfectly accurate.
- Ambiguous messages remain difficult to classify.
- Gemini introduces external API dependency, latency, and potential usage costs.
- The current 70% routing threshold is a practical configuration and requires systematic validation/calibration.
- The complete application architecture is not yet equivalent to a deployed emergency-response platform.
- The current model primarily handles English text.
- AI-generated outputs should not be treated as a replacement for human decision-making in critical rescue situations.

9.3 Safety Considerations

RESQAI is intended to support information organization and disaster response rather than independently make life-critical decisions.

For a real deployment:

- High-risk incidents should have human verification.
- AI confidence should be visible to authorized operators.
- Personal and location information should be handled securely.
- API keys and credentials must never be exposed in source code or public repositories.
- The system should maintain logs and audit trails for important incident decisions.
- False positives and false negatives should be considered during emergency operations.

10. Future Scope

The following improvements can extend RESQAI into a complete humanitarian response platform.

10.1 User-Facing Interface

A simple reporter interface can allow users to:

- Submit an SOS message.
- Share or select a location.
- View incident status.
- Receive relevant safety information.

10.2 Administrator Dashboard

The administrator dashboard can provide:

- Active incident monitoring.
- Category and urgency filtering.
- Incident map visualization.
- Rescue-team assignment.
- Status monitoring.
- Incident reporting and analytics.

10.3 Rescue-Team Dashboard

Rescue teams can receive assigned tasks containing:

- Incident location.
- Incident category.
- Urgency level.
- Original message.
- Recommended action.
- Current incident status.

Possible status transitions include:

Assigned → Accepted → En Route → On Site → Resolved

10.4 Map Integration

A map service can be integrated to display incident locations and assist rescue teams with navigation.

10.5 Multilingual Disaster Analysis

Future versions could support multiple Indian and international languages to make the system accessible during regional disasters.

10.6 Improved Urgency Detection

Urgency can be modeled more systematically using dedicated classification or ranking techniques.

10.7 Real-Time Data Streams

The system can eventually process live streams from social-media platforms, emergency channels, or other authorized information sources.

10.8 Continuous Model Improvement

New labeled disaster messages can be incorporated into periodic retraining. Model performance can then be monitored using validation and test benchmarks.

11. Conclusion

RESQAI presents an AI-assisted approach to humanitarian disaster information classification. The current prototype uses the HumAID dataset and a fine-tuned BERTweet model to classify disaster-related messages into ten humanitarian categories.

The final BERTweet classifier achieved 78.67% test accuracy and 78.27% weighted F1-score. A hybrid architecture was subsequently developed in which BERTweet serves as the primary classifier and Gemini provides contextual analysis for low-confidence or `not_humanitarian` cases.

In addition to the machine learning pipeline, this report defines the software engineering design of the proposed RESQAI platform through requirements analysis, Use Case, UML architecture, Activity, Class, and SDLC diagrams.

The system provides a foundation for a broader disaster-response platform involving reporter interfaces, administrative monitoring, rescue-team coordination, location services, and structured humanitarian information. Further experimentation is required to improve classification accuracy and formally evaluate the end-to-end hybrid system.

Bibliography

- [1] D. Q. Nguyen, T. Vu, and A. T. Nguyen, “BERTweet: A pre-trained language model for English Tweets,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.
- [2] HumAID: Humanitarian Dataset for Information Classification, Humanitarian information extraction and classification research dataset.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of NAACL-HLT*, 2019.
- [4] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” *Proceedings of EMNLP: System Demonstrations*, 2020.
- [5] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems*, 2019.
- [6] Object Management Group, “Unified Modeling Language (UML) Specification.”
- [7] I. Sommerville, *Software Engineering*, Pearson.